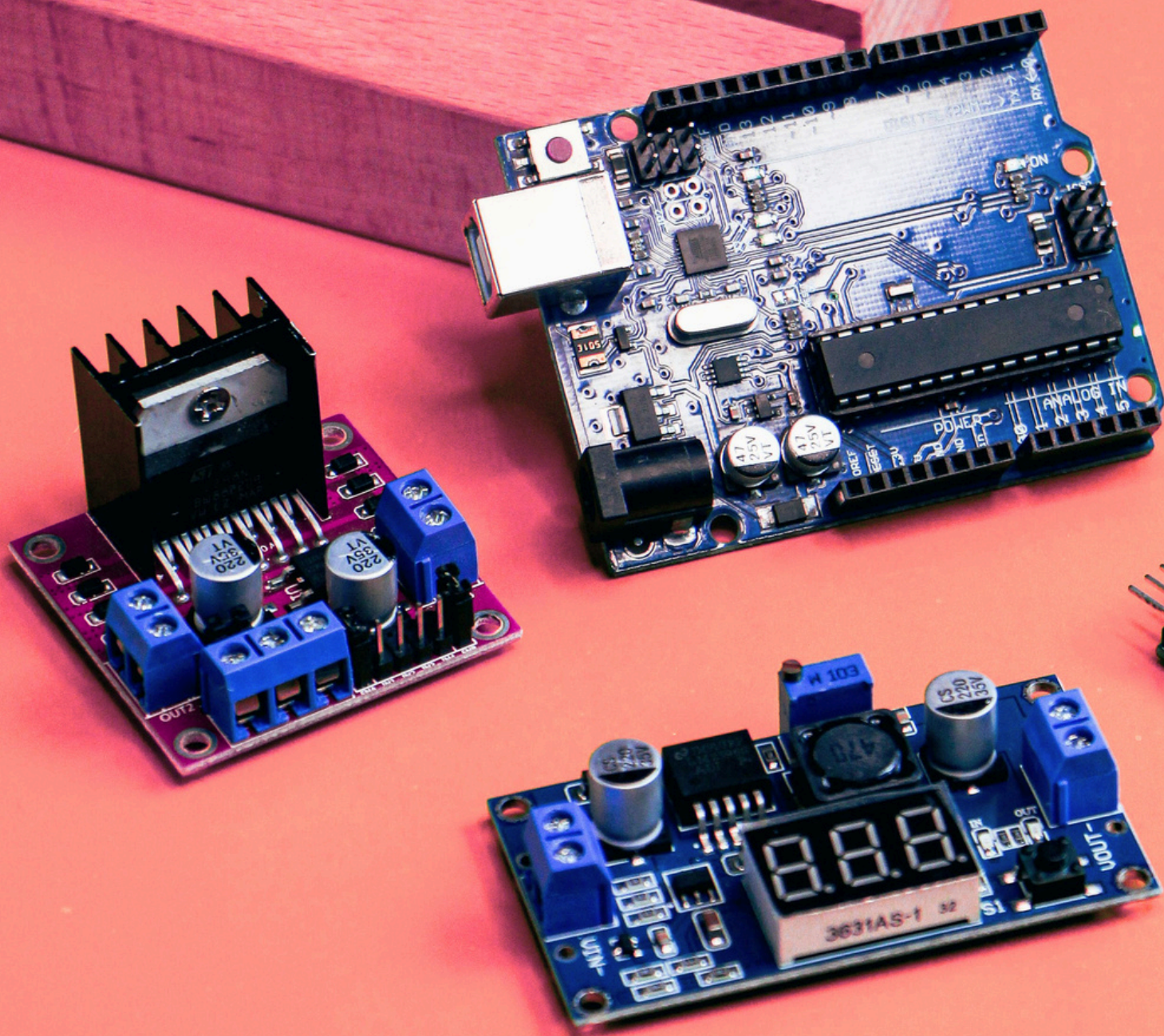


CAPITULO 1

1

A CRIAÇÃO E A UTILIZAÇÃO DO ARDUINO



O QUE É UM ARDUINO?

Um Arduino nada mais é que do que uma plataforma de desenvolvimento de projetos eletrônicos, constituída tanto de **hardware** quanto de **software**.

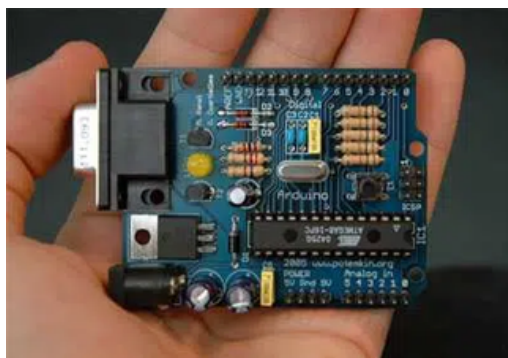
A parte do **hardware** e placa do Arduino tem como componente principal um microcontrolador, um tipo de processador bem menor que o normal, e tem o objetivo de controlar sistemas eletrônicos, como sensores e motores.

O Arduino é um placa open source, ou seja, todos tem acesso a propriedade intelectual da placa, fazendo os usuários realizarem melhorias ao código para o aprimoramento da plataforma.

HISTÓRIA DO ARDUINO

Antes dos anos 2000, o desenvolvimento com microcontroladores era dominado por plataformas como o BASIC Stamp, que, apesar de útil, apresentava limitações importantes. Seu custo era relativamente alto, o ambiente de programação funcionava apenas em Windows e seu uso era mais voltado para pessoas com conhecimento técnico. Isso dificultava o acesso de estudantes, artistas e iniciantes interessados em explorar a tecnologia de forma criativa.

Diante desse cenário, um grupo formado por Hernando Barragán, Massimo Banzi, Casey Reas e Ben Fry buscou criar uma alternativa mais acessível. Barragán desenvolveu o Wiring como parte de sua tese, enquanto Reas e Fry criaram o Processing, um ambiente de programação voltado à arte. A partir dessas ideias, surgiu o Arduino, com o objetivo de ser uma plataforma de baixo custo, fácil de usar e adequada tanto para iniciantes quanto para projetos criativos.



Arduino Board Serial, uma das primeiras placas disponíveis comercialmente.

<https://embarcados.com.br/wp-content/uploads/2023/05/image-26.png.webp>

Um dos grandes diferenciais do Arduino foi sua filosofia de código aberto. Tanto o hardware quanto o software foram disponibilizados livremente, permitindo que qualquer pessoa pudesse estudar, modificar e reproduzir as placas. Isso contribuiu para o rápido crescimento da comunidade e para a popularização da plataforma, mesmo com o surgimento de clones no mercado.

Em 2008, foi criada a Arduino LLC para gerenciar a marca e sua propriedade intelectual. Apesar de desafios, como disputas internas entre os

HARDWARE:

Os hardwares são as peças físicas que compõem um computador, como as placas, o monitor, o teclado, a placa-mãe e o disco rígido.

<https://www.todamateria.com.br/hardware-e-software/>

SOFTWARE

Software é todo programa ou conjunto de instruções que orienta o funcionamento de dispositivos eletrônicos.

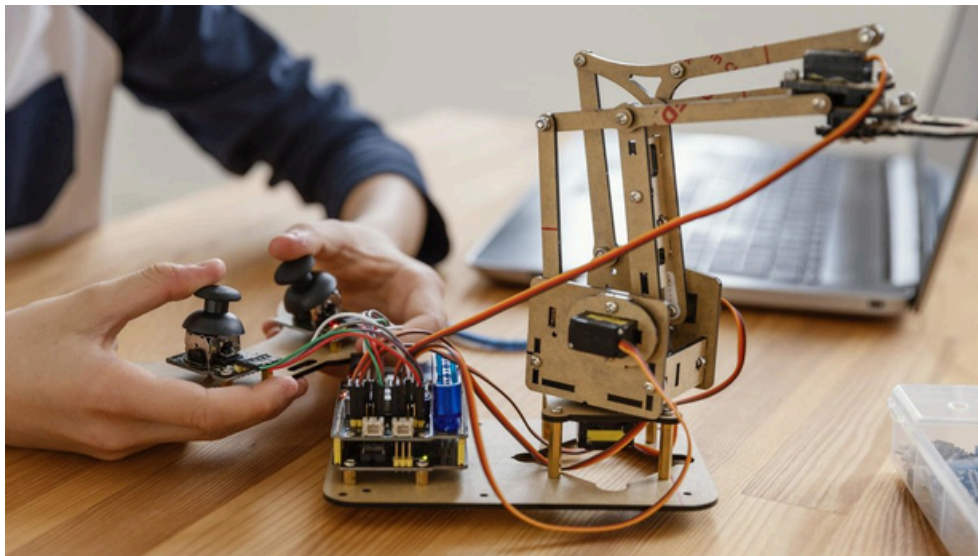
<https://tecnoblog.net/responde/o-que-e-software/>



<https://victorvision.com.br/wp-content/uploads/2021/10/o-que-e-arduino.jpg>

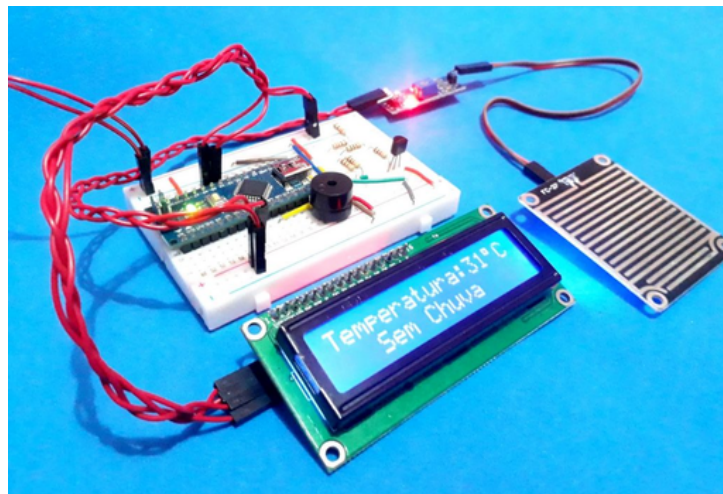
ONDE É UTILIZADO

O arduino é uma controladora poderosa em prototipagem de produtos e projetos pessoais, por seu baixo custo, facilidade de montagem e conhecimento acessível ele é muito usado por engenheiros e makers para transformar ideias em realidade, como em estufas automáticas que regam plantas sozinhas, em alarmes caseiros, em semáforos de feira de ciências e até em instrumentos musicais eletrônicos.



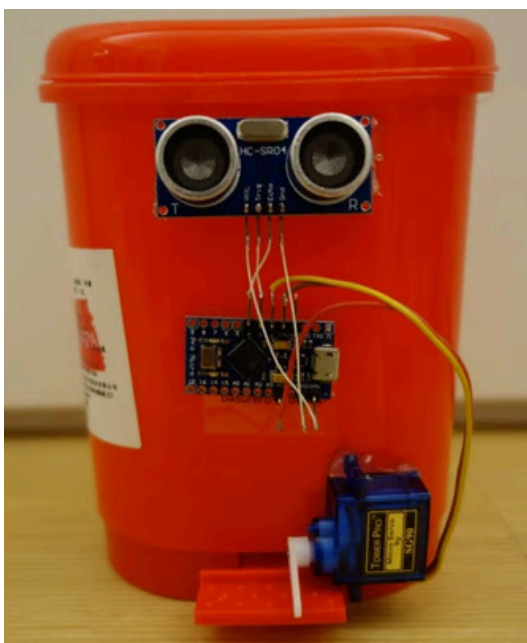
Braço robótico com arduino.

<https://victorvision.com.br/blog/projeto-com-arduino/>



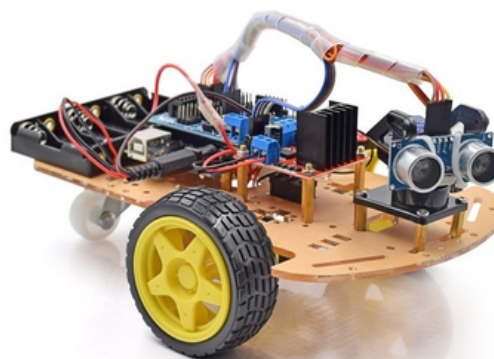
Sensor de chuva automático com arduino.

<https://www.usinainfo.com.br/blog/asensor-de-chuva-arduino-em-alarme-para-automacao-residencial/>



Lixeira automática arduino.

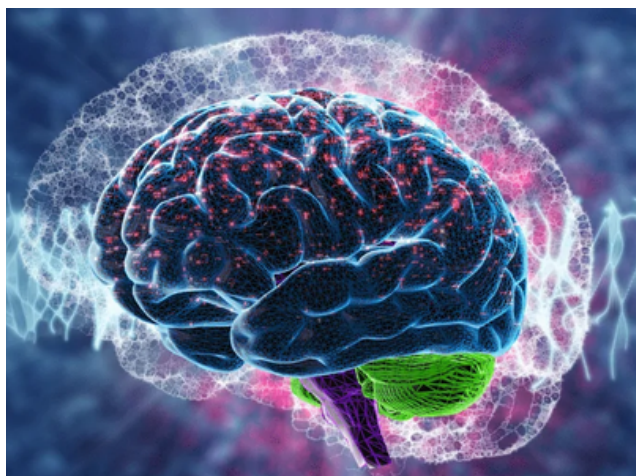
<https://victorvision.com.br/blog/projeto-com-arduino/>



Carrinho inteligente com arduino.

<https://guiarobotica.com/projetos-com-arduino/>

PENSAMENTO COMPUTACIONAL



O pensamento computacional é a capacidade de resolver problemas de forma lógica, organizada e sistemática, utilizando conceitos como decomposição, reconhecimento de padrões, abstração e criação de algoritmos. Ele não se limita à programação, mas envolve uma maneira de pensar que pode ser aplicada em diversas áreas do conhecimento, ajudando na tomada de decisões e na construção de soluções eficientes.

No contexto educacional, o desenvolvimento do pensamento computacional é considerado essencial. A Base Nacional Comum Curricular (BNCC) destaca a importância das tecnologias digitais e da cultura digital na formação dos estudantes, incentivando o uso de ferramentas que promovam o raciocínio lógico, a criatividade e a resolução de problemas. Dessa forma, trabalhar esse tipo de pensamento nas escolas prepara os alunos para os desafios do mundo contemporâneo.

Nesse cenário, o Arduino se apresenta como uma ferramenta poderosa para o desenvolvimento do pensamento computacional. Por meio da criação de projetos práticos, os estudantes precisam analisar problemas, dividir tarefas, testar soluções e corrigir erros, colocando em prática conceitos fundamentais da computação. Além disso, o caráter interativo e experimental do Arduino torna o aprendizado mais dinâmico e significativo, aproximando a teoria da prática e estimulando a autonomia dos alunos.



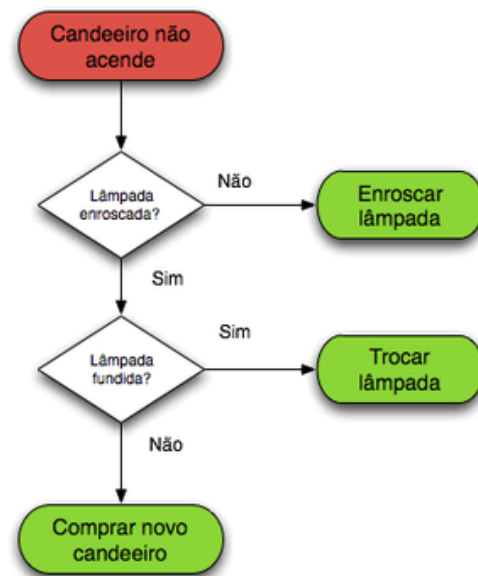
Três eixos do pensamento computacional
<https://www.computacional.com.br/educacao-basica>

CONCEITOS FUNDAMENTAIS

O pensamento computacional pode ser aplicado de forma prática por meio de conceitos fundamentais. A decomposição consiste em dividir um problema maior em partes menores, como ao criar um sistema de alarme residencial separando-o em sensor de presença, sirene e sistema de senha. Isso facilita o entendimento e a implementação de cada etapa do projeto.

O reconhecimento de padrões ocorre quando se percebe que diferentes sensores funcionam de maneira semelhante, utilizando a mesma lógica de leitura de dados, o que permite reaproveitar códigos e soluções. Já a abstração envolve ignorar detalhes mais complexos do funcionamento interno dos componentes, como o microcontrolador, para focar na lógica principal do projeto.

Por fim, os algoritmos representam a criação de um passo a passo lógico para resolver um problema, como programar um sensor de umidade do solo para acionar automaticamente a irrigação de uma planta quando necessário. Esses conceitos tornam o uso do Arduino uma ferramenta eficaz para desenvolver o raciocínio lógico e a resolução de problemas.



exemplo da divisão de etapas por algoritmos

PINOS E ENERGIA

PINO DIGITAL E ANALÓGICO

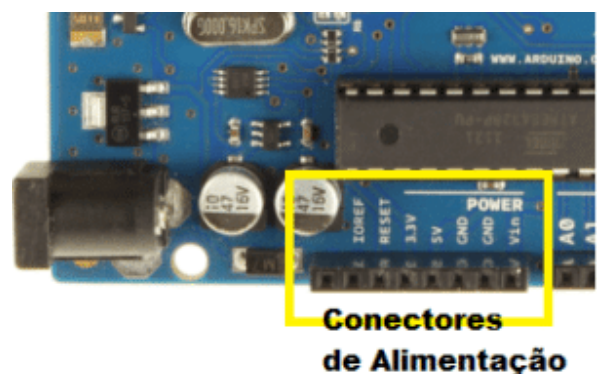
No arduino, os pinos digitais e analógicos são usados para se comunicar com sensores, motores e outras coisas, mas é importante saber a diferença. Os pinos digitais só têm dois estados, ligado e desligado, o que pode ser útil, por exemplo, para LEDs e motores. Já os analógicos servem para medir valores variáveis, permitindo que o Arduino leia informações mais detalhadas, sendo úteis para joysticks e giroscópios, por exemplo.



CONECTORES DE ALIMENTAÇÃO

GND vem do inglês **Ground**, que significa chão ou terra. Ele é o ponto de referência de todo o circuito, o "zero" de tudo. A energia elétrica precisa sair de um lugar e voltar para outro para funcionar, e o GND é esse ponto de retorno.

Os pinos que fornecem energia para os componentes são os de 5V e o de 3.3V. O pino 5V entrega uma tensão de cinco volts e é o mais usado no dia a dia, ideal para alimentar componentes simples. Já o 3.3V entrega uma tensão menor e é usado com componentes mais sensíveis. É importante sempre verificar qual tensão o seu componente aceita antes de conectar.



CAPITULO 2

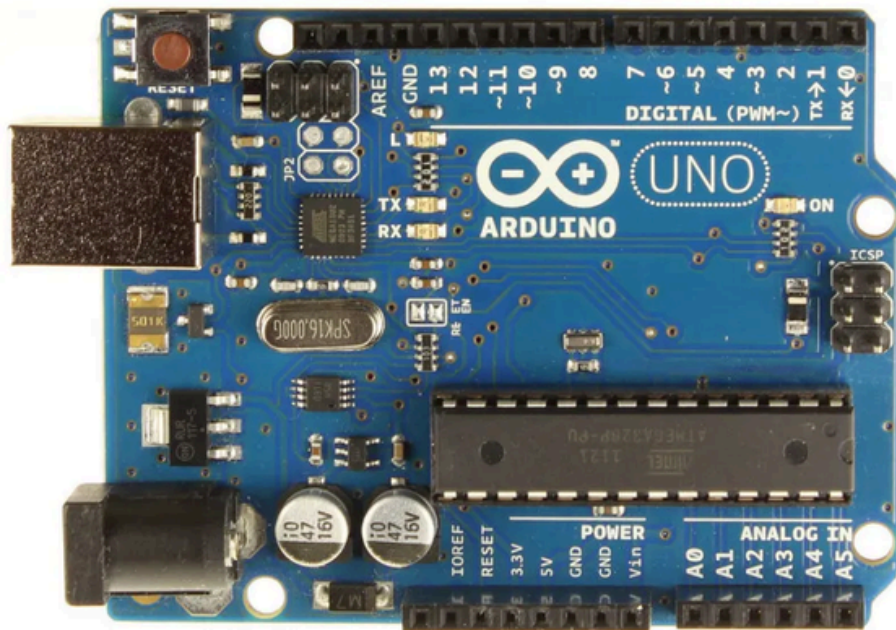
2

ANATOMIA DO KIT E INTRODUÇÃO A PROGRAMAÇÃO



ANATOMIA DO LABKIT

Vamos conhecer um pouco dos componentes compatíveis com o Arduino inclusos no LABKIT:



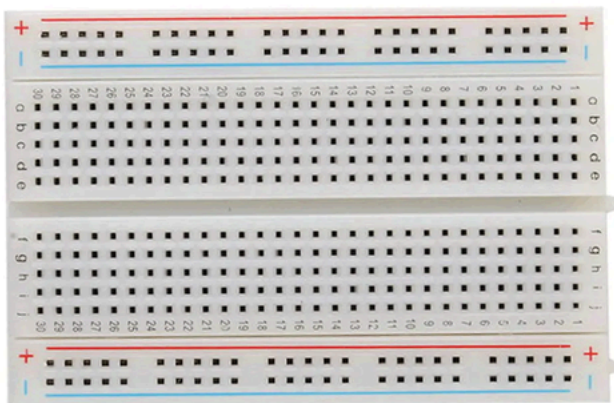
A placa do arduino UNO



Luzes LED



Cabos Jumper



Protoboard



HC-SR04 (sensor ultrassônico)

ANATOMIA DO LABKIT

Vamos conhecer um pouco dos componentes compatíveis com o Arduino inclusos no LABKIT:



Display de 7 segmentos



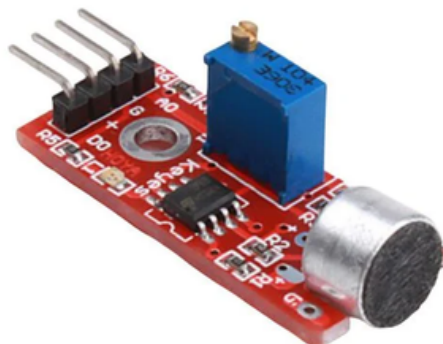
LCD 16x2



Amplificador PAM8403



Buzzer ativo



KY-037 (sensor de som)

INTRODUÇÃO A PROGRAMAÇÃO

Antes de começar a programar é importante lembrar que o arduino não tem nenhum tipo de inteligência, ele apenas vai fazer tudo que você mandar da forma que você mandar.

Agora sim vamos dar nossos primeiros passos da programação com arduino. Todo código de arduino começa com dois blocos obrigatórios: o **setup()** e o **loop()**. O setup roda uma única vez, logo quando o arduino é ligado, e serve para fazer as configurações iniciais do programa. Já o loop roda para sempre, repetindo sem parar enquanto o Arduino estiver ligado, e é onde a maior parte do programa acontece.

```
1 void setup() {  
2   // aqui você configura as coisas uma vez só  
3 }  
4  
5 void loop() {  
6   // aqui o Arduino fica repetindo pra sempre  
7 }
```

GUARDANDO INFORMAÇÕES

Nos sempre temos que registrar informações em nossa memória, na programação do arduino, quem tem esse papel da memória são as variáveis, elas são basicamente uma caixinha com nome que guarda um valor dentro.

Para criar uma variável no arduino, você precisa dizer o tipo, dar um nome e atribuir um valor. O tipo mais comum é o **int**, usado para guardar números inteiros, e o **bool**, usado para guardar apenas dois estados: verdadeiro (**true**) ou falso (**false**).

```
1 int idade = 15;  
2 int velocidade = 200;  
3 bool luzAcesa = false
```

EXECUTANDO COMANDOS

Para fazer o arduino executar ordens usamos funções, que são comandos prontos que o arduino já conhece e sabe executar.

O primeiro comando que você precisa conhecer é o **pinMode()**. Ele serve para dizer ao Arduino se um pino vai mandar sinal ou receber sinal. Você passa o número do pino e o modo, que pode ser **OUTPUT** para saída ou **INPUT** para entrada.

```
1 pinMode(13, OUTPUT); // o pino 13 vai mandar sinal  
2 pinMode(13, INPUT); // o pino 13 vai receber sinal
```

Depois de configurar o pino, você usa o **digitalWrite()** para de fato ligar ou desligar ele. O valor **HIGH** significa ligado e o valor **LOW** significa desligado.

```
1 digitalWrite(13, HIGH); // liga o pino 13
2 digitalWrite(13, LOW);  // desliga o pino 13
```

Por último, o **delay()** faz o Arduino pausar por um tempo antes de continuar. O valor é em milissegundos, então **delay(1000)** significa uma pausa de exatamente um segundo.

```
1 delay(1000); // espera 1 segundo
```

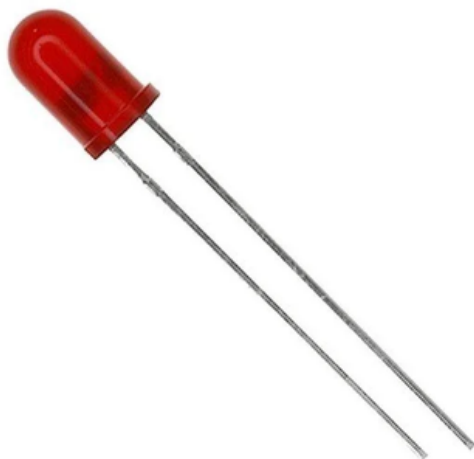
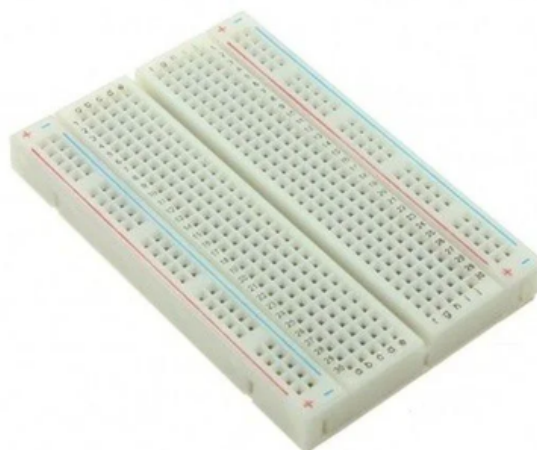
Juntando os três, você já consegue fazer o Arduino ligar um LED, esperar um segundo e desligar.

```
1 digitalWrite(13, HIGH); // liga
2 delay(1000);           // espera 1 segundo
3 digitalWrite(13, LOW);  // desliga
4 delay(1000);           // espera mais 1 segundo
```

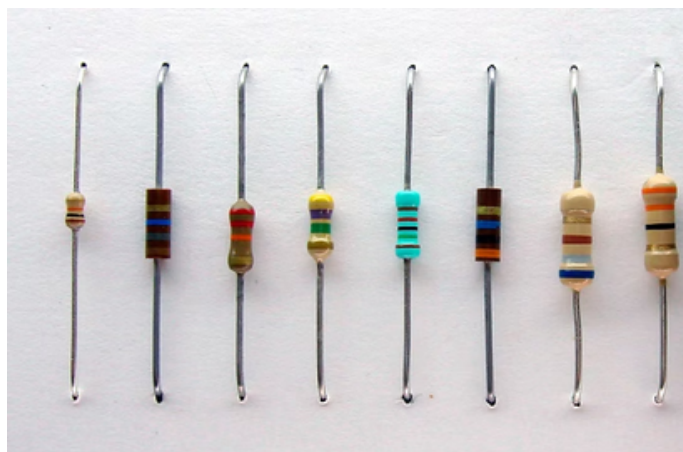
APRENDENDO A CONSTRUIR CIRCUITOS

Antes de começar a programar é necessário montar o seu circuito, por isso vamos ver peças e componentes essenciais para montar seu primeiro circuito.

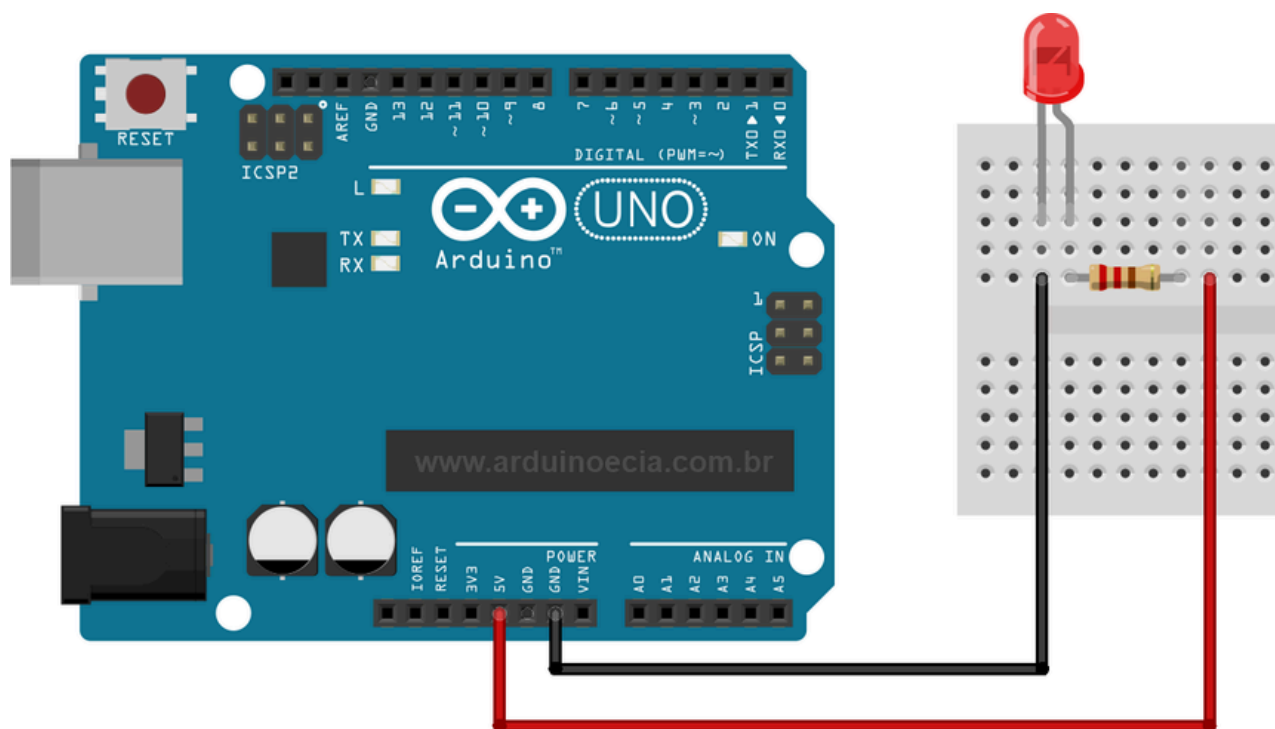
A primeira é a **protoboard**, também chamada de matriz de contato. Ela é uma placa cheia de furinhos por onde você encaixa os componentes sem precisar de solda. As fileiras do meio são ligadas na horizontal, ou seja, tudo que você encaixar na mesma fileira está conectado. Já as colunas das bordas, marcadas com + e -, são ligadas na vertical e servem para distribuir energia pelo circuito.



O segundo componente é o **LED**, que é uma luz que acende quando passa corrente elétrica por ela. O **LED** tem polaridade, ou seja, tem um lado certo para conectar. A perninha mais longa é o positivo, e a mais curta é o negativo, que deve ser ligado ao **GND**.



O terceiro é o **resistor**. Ele serve para limitar a quantidade de corrente que passa pelo LED, protegendo o componente de queimar. Sempre que usar um LED, use um resistor junto, sem exceção.



Exemplo de LED conectado a uma placa protoboar e um arduino
<https://arduinoecia.com.br/codigo-de-cores-de-resistores/>

CAPITULO 3

3

PENSAMENTO COMPUTACIONAL E ATIVIDADES



AGORA É SUA VEZ

Imagine que você foi contratado pela Secretaria de Mobilidade Urbana de Criciúma para resolver um problema grave de segurança viária. No cruzamento próximo à nossa escola SESI, o fluxo de pedestres aumentou muito devido à saída dos turnos de aula, e os acidentes aumentaram porque não há um controle de tráfego automatizado.

O seu desafio técnico é: Projetar e programar um protótipo de semáforo utilizando a plataforma Arduino. O sistema deve alertar motoristas e pedestres de forma clara, cíclica e sem falhas, garantindo a transição segura entre os estados de "Siga" (Verde), "Atenção" (Amarelo) e "Pare" (Vermelho).



Como engenheiro(a) responsável, você precisará usar o Pensamento Computacional para dividir esse problema (Decomposição), identificar as repetições de comandos (Padrões), focar no comportamento lógico das luzes (Abstração) e escrever as regras exatas para o hardware obedecer (Algoritmo).

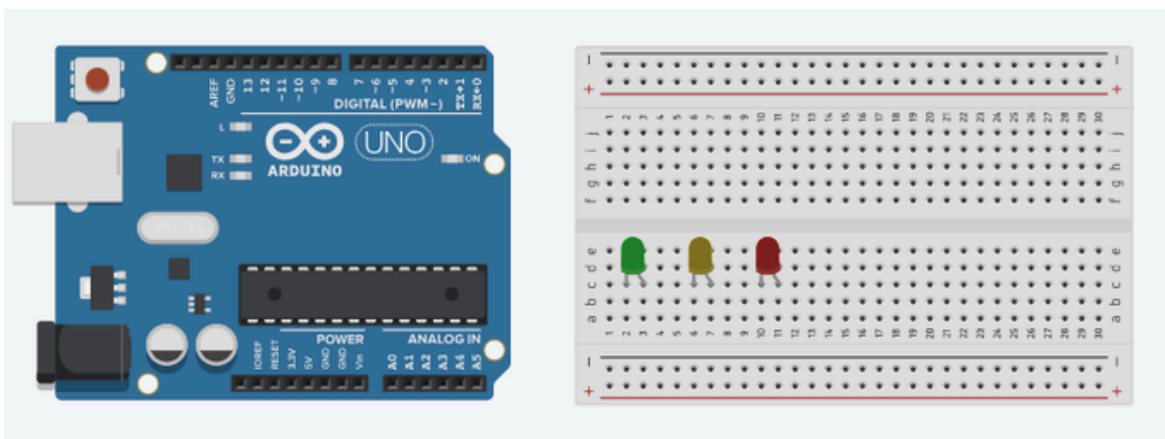
1 Para construção do trabalho, vamos começar pela montagem do circuito utilizando os seguintes componentes:

- Arduino UNO;
- Protoboard;
- 3 LEDs (1 verde, 1 amarelo e 1 vermelho);
- Jumpers
- Resistores

Agora vamos começar a montagem do circuito, certifique-se de que o arduino está desconectado do computador, e siga as instruções:

1. Insira os três LEDs (Verde, Amarelo e Vermelho) na vertical da protoboard, deixando algumas colunas de espaço entre cada um

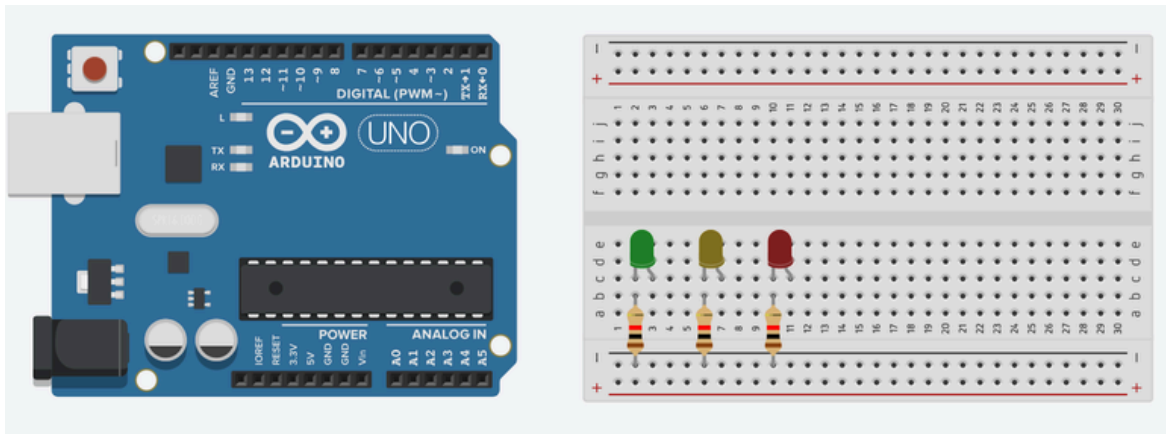
A perna mais longa é o (+), encaixe ela voltada para o lado direito da protoboard já a mais curta é o (-), encaixe ela voltada pro lado esquerdo da protoboard.



2. Pegue um resistor de 220Ω para cada LED

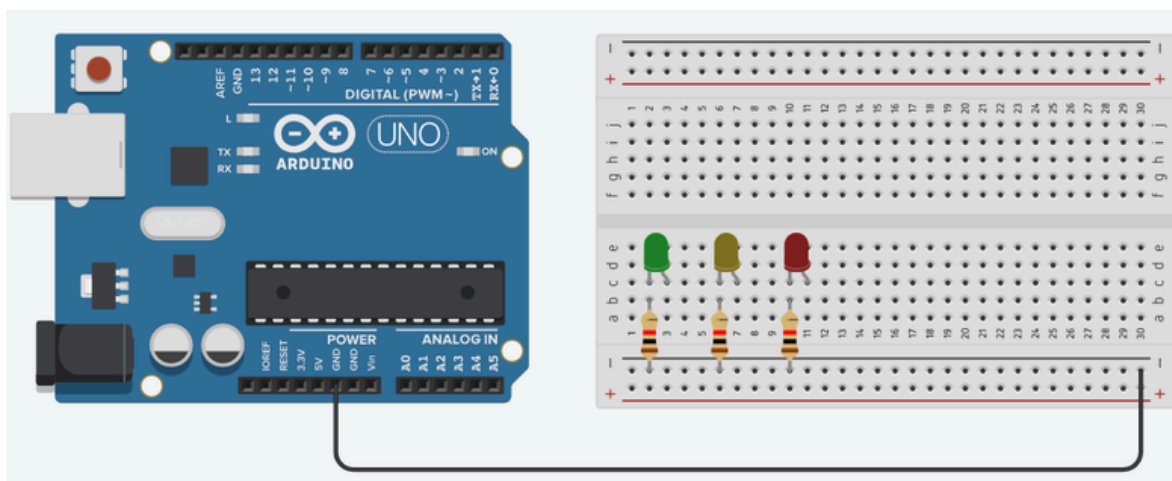
Conecte uma ponta do resistor na mesma linha da perna curta (-) do LED

Conecte a outra ponta na barra lateral azul da protoboard (linha do negativo comum)



3. Pegue um cabo jumper

Conecte-o da barra lateral azul da protoboard até o pino GND do Arduino

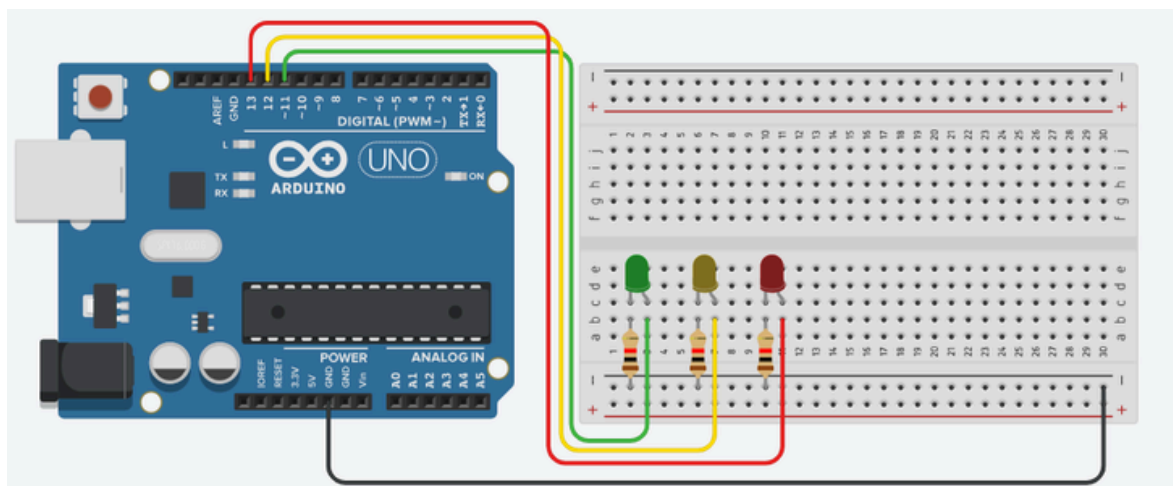


4. Por ultimo, utilize um jumper para conectar?

Perna longa (+) do LED Verde → Pino Digital 11

Perna longa (+) do LED Amarelo → Pino Digital 12

Perna longa (+) do LED Vermelho → Pino Digital 13



2

Agora para fazer o protótipo funcionar precisamos programar o código do semáforo, com base nos conhecimentos que você adquiriu ao longo do capítulo anterior, complete as partes vermelhas do código a seguir dentro do VS Code:

```
// Definição das portas lógicas do nosso hardware
__ LED_VERDE    = 11; // Como podemos criar variáveis no arduino?
__ LED_AMARELO  = 12;
__ LED_VERMELHO = 13;

void setup() {
    // Configurando os pinos dos LEDs como saídas de sinal
    pinMode(LED_VERDE, ____); //O LEDs vão receber ou enviar informações?
    pinMode(LED_AMARELO, ____);
    pinMode(LED_VERMELHO, ____);
}

void ____() { // essa operação vai ser única ou contínua?
    //VERDE - Fluxo Liberado
    digitalWrite(LED_VERDE, HIGH); // Liga o LED Verde
    ____ (3000);                    // Qual o comando que vai manter ele ligado?
    digitalWrite(LED_VERDE, ____); // O que deve acontecer com o LED Verde aqui?

    // AMARELO
    digitalWrite(____, HIGH); // Qual LED deve acender agora?
    ____ (1000);               // Qual o comando que vai manter ele ligado?
    digitalWrite(LED_AMARELO, ____); // Desliga o LED Amarelo

    // VERMELHO - Perigo, Pare!
    digitalWrite(LED_VERMELHO, ____); // O que ativa a energia do LED Vermelho?
    ____ (3000);                      // Qual o comando que vai manter ele ligado?
    digitalWrite(LED_VERMELHO, ____); // Desliga o LED Vermelho
```